

Solutions for Homework Assignment #4

Answer to Question 1.

a. The complete line 3 is: “ $x := \text{POWEROFTEHTOBINARY}(n/2)$ ”. According to the specification of `POWEROFTEHTOBINARY`, this gives the binary representation of $10^{n/2}$, so that line 4 computes and returns the binary representation of $10^{n/2} \times 10^{n/2} = 10^n$, which is what we want. The running time of the algorithm is described by the following recurrence:

$$T(n) = T(n/2) + cn^{\log_2 3} \quad (1)$$

This is because the computation involves the following steps:

- (a) One recursive call to `POWEROFTEHTOBINARY` on input $n/2$, resulting in the term $T(n/2)$.
- (b) A call to `FASTMULT` on inputs x and x , where x is the binary representation of $10^{n/2}$. So, the length of x (in bits) is $\lceil \log_2 10^{n/2} \rceil = \frac{n}{2} \log_2 10 \leq kn$, for some constant k . The call to `FASTMULT` therefore takes $\Theta((kn)^{\log_2 3}) = \Theta(n^{\log_2 3})$ time, resulting in the term $cn^{\log_2 3}$.

Equation (1) is the divide-and-conquer recurrence with $a = 1$, $b = 2$, and $d = \log_2 3$. Since $a < b^d$, by the Master Theorem the running time of `POWEROFTEHTOBINARY` is $T(n) = \Theta(n^{\log_2 3})$.

b. The complete line 6 is:

$x := \text{SUM}(\text{FASTMULT}(\text{DECIMALTOBINARY}(x_L), \text{POWEROFTEHTOBINARY}(n/2)), \text{DECIMALTOBINARY}(x_R))$

By the definition of x_L and x_R we have that $x = x_L \cdot 10^{n/2} + x_R$, and the right-hand-side is precisely what is computed above. (Here we are abusing notation slightly by using the decimal strings x_L and x_R for the numbers they represent.) The running time of this algorithm is described by the following recurrence, where n is the number of digits in the input x :

$$T(n) = 2T(n/2) + cn^{\log_2 3} \quad (2)$$

This is because the computation involves the following steps:

- (a) Splitting x into x_L and x_R can be done in $\Theta(1)$ time (if we modify the algorithm to use indices as parameters) or $\Theta(n)$ time if we naively copy the two halves of x into x_L and x_R .
- (b) Two recursive calls to `DECIMALTOBINARY` on inputs x_L and x_R that have $n/2$ digits each, resulting in the term $2T(n/2)$.
- (c) A call to `POWEROFTEHTOBINARY` on input $n/2$. By Part (a), this takes $\Theta((n/2)^{\log_2 3}) = \Theta(n^{\log_2 3})$ time.
- (d) A call to `FASTMULT` on inputs the binary representations of x_L and $10^{n/2}$. These are binary representations of $n/2$ -digit numbers and therefore have length at most $\lceil \log_2 10 \rceil n = kn$ bits, for some constant k . This call therefore takes $\Theta((kn)^{\log_2 3}) = \Theta(n^{\log_2 3})$ time.
- (e) A call to `SUM` with inputs the binary representations of $x_L \cdot 10^{n/2}$ and x_R . These are binary representations of an n -digit and an $n/2$ -digit number, so they have lengths at most $\lceil \log_2 10 \rceil n$ and $\lceil \log_2 10 \rceil n/2$ bits, respectively. Since both inputs to `SUM` have length at most kn for some constant k , this call takes $\Theta(n)$ time.

The time required for steps (c) and (d) dominates the time required for steps (a) and (e), yielding recurrence (2). This is a divide-and-conquer recurrence with $a = 2$, $b = 2$, and $d = \log_2 3$. Since $a < b^d$, by the Master Theorem, the running time of the algorithm `DECIMALTOBINARY` is $T(n) = \Theta(n^d) = \Theta(n^{\log_2 3})$.

Answer to Question 2.

ALGORITHM DESCRIPTION AND CORRECTNESS: Let ℓ and r be such that $1 \leq \ell \leq r \leq n$, so $A[\ell..r]$ is a subarray of A . Define the **maximum value prefix** of $A[\ell..r]$ to be the maximum possible value of a subarray of $A[\ell..r]$ that starts at ℓ ; i.e., $\max_{\ell \leq k \leq r} \text{val}(A[\ell..k])$; and the **maximum value suffix** of $A[\ell..r]$ to be the maximum possible value of a subarray of $A[\ell..r]$ that ends at r ; i.e., $\max_{\ell \leq k \leq r} \text{val}(A[k..r])$.

Now, suppose A is split into two halves, left and right. (If n is odd, one “half” has one more element than the other.) Consider a maximum value subarray $A[i..j]$ of A . There are three possibilities:

- (a) $A[i..j]$ is entirely within the left half of A ; i.e., $1 \leq i \leq j \leq m$, where $m = (\ell + r) \text{ div } 2$ (the midpoint of ℓ and r);
- (b) $A[i..j]$ is entirely within the right half of A ; i.e., $m < i \leq j \leq n$; or
- (c) $A[i..j]$ “straddles” the two halves of A , i.e. $1 \leq i \leq m < j \leq n$.

We can deal with possibilities (a) and (b), by simply applying the algorithm recursively to the first and second half of A . For possibility (c) we note that, in this case, the maximum value subarray consists of a maximum value suffix of the first half of A and a maximum value prefix of the second half of A . (This can be shown easily by contradiction, using a “cut-and-paste” argument.) Therefore, to deal with possibility (c), we construct our recursive algorithm so that it returns not only the maximum value of a subarray, but also the maximum value of a prefix and the maximum value of a suffix. In addition, to compute efficiently the maximum value of a prefix or suffix, we make the algorithm return the value of the entire subarray $A[\ell..r]$.

To express the recursive algorithm conveniently, we write it as operating on an arbitrary subarray $A[\ell..r]$ of A . The call $\text{RECMAXVAL}(A, \ell, r)$, where $1 \leq \ell \leq r \leq n$, returns a tuple (v, p, s, b) , where

- v is the maximum value of a subarray of $A[\ell..r]$;
- p is the maximum value of a prefix of $A[\ell..r]$;
- s is the maximum value of a suffix of $A[\ell..r]$; and
- t is the total value of $A[\ell..r]$.

```

RECMAXVAL( $A, \ell, r$ )
if  $\ell = r$  then return ( $A[\ell], A[\ell], A[\ell], A[\ell]$ )
else
     $m := (\ell + r) \text{ div } 2$ 
    ( $v_L, p_L, s_L, t_L$ ) := RECMAXVAL( $A, \ell, m$ )
    ( $v_R, p_R, s_R, t_R$ ) := RECMAXVAL( $A, m + 1, r$ )
     $v := \max(v_L, v_R, s_L + p_R)$ 
     $p := \max(t_L + p_R, p_L)$ 
     $s := \max(s_L + t_R, s_R)$ 
     $t := t_L + t_R$ 
    return ( $v, p, s, t$ )

```

The correctness of this algorithm (i.e., the fact that it returns a tuple where each component has the value specified above) follows by the preceding discussion. (More formally, we use induction on the length of the subarray $A[\ell..r]$, but the preceding discussion is the core of the argument for the induction step.)

To determine the maximum value of a subarray of A we simply run the following algorithm:

```

MAXVAL( $A$ )
( $v, p, s, b$ ) := RECMAXVAL( $A, 1, n$ )
return  $v$ 

```

RUNNING TIME: The running time of $\text{RECMAXVAL}(A, \ell, r)$ is described by the recurrence $T(k) = 2T(k/2) + c$, where k is the length of subarray $A[\ell..r]$, i.e., $k = r - \ell + 1$. This is a divide-and-conquer recurrence with $a = 2$, $b = 2$, and $d = 0$. Since $a > b^d$, by the Master Theorem the running time of RECMAXVAL is $T(k) = \Theta(k^{\log_2 2}) = \Theta(k)$. So, the running time of MAXVAL is dominated by the call to RECMAXVAL , and it is $\Theta(n)$.